

Agentic RL 时代的 Infra 重构：以 Forge、ROLL、Seer、Slime 为例



低级炼丹师

收录于 · 大模型/深度学习 >

538 人赞同了该文章 >

这篇文章是在阅读了 Forge、ROLL、Seer、[slime+](#) 的博客及技术报告（链接在最后）之后，对这四家公司在 infra 和 agentic RL 相关工作中的部分底层算法所做的整理与总结。作者主要是纯算法背景，对 infra 也有一定了解，但并不算特别深入。因此，这篇文章更多是从应用算法的视角出发，尝试理解和梳理 RL infra 相关技术，并且作为个人学习相关知识的笔记。文中若有理解不准确或表述不严谨之处，也非常欢迎各位老师的批评指正。

过去一段时间，强化学习在大语言模型上的突破，主要集中在 reasoning 场景。无论是数学、代码还是一般推理任务，大家熟悉的训练方式大体都是一致的：模型生成一段完整回答，系统根据最终结果给出奖励，再进行参数更新。即使这些回答已经足够长、足够复杂，它们本质上还是一种“单次生成”问题，模型的核心目标仍然是把一次 response 写对。

但 Agentic RL 让这个问题变了。模型不再只是生成一个答案，而是要在环境中持续行动：调用工具、接收观察、管理上下文，并在多轮交互后逐步完成任务。[MiniMax+](#) 在 Forge 的博客里把这个变化说得很直接：在真实世界的大规模复杂场景里做 RL，核心难题始终是如何在系统吞吐量、训练稳定性与 Agent 灵活性三者之间取得平衡。阿里在 ROLL 的论文里也用了一句很形象的话来概括这种区别：“[RLVR+](#) 训练的是一个‘会回答’的模型，而 Agentic RL 训练的是一个‘会行动’的模型——跨时间、跨状态、跨不确定性地行动。”

也正因为如此，Agentic RL 面对的问题不再只是长序列训练，而是整套系统问题：Agent 脚手架怎么接入，环境如何管理，多轮交互里的长尾 rollout 怎么调度，训练和部署怎么保持一致，奖励与测试如何避免被“投机取巧”。最近几家代表性的系统——MiniMax 的 Forge、阿里的 ROLL、Moonshot 的 Seer、智谱的 [slime](#)——虽然角度不同，但其实都在回答同一个问题：当我们开始训练“会行动”的模型时，RL 基础设施应该如何重构？

0. 分析框架：Agentic RL Infra 在优化什么？

Forge 博客在开篇给出了一个非常适合作为总纲的形式化目标。它把 Agent RL 系统抽象成“最大化有效训练收益”：

$$\max_{\theta} J(\theta) = \text{Throughput}(\mathcal{A}) \times \text{Sample Efficiency}(\mathcal{A})$$

同时满足：

系统能支持任意 Agent：

$$\forall \mathcal{A} \in \Omega_{\text{agent}}$$

更新方差受控：



$$\mathbb{E}[\|J^{(T)} - J^*\|] < \epsilon$$

Forge 原文里对这个公式后面的解释其实非常清楚。它指出，Throughput 指的是每秒处理的原始 token 数量，主要受 RL 系统中四部分控制：Rollout、Training、Data Processing 和 I/O。而 Sample Efficiency 指的是每个样本带来的平均性能提升，它取决于数据分布、数据质量、算法效率以及 off-policy 程度。稳定性和收敛性则需要通过训练过程中的监控指标来判断。

这个定义有一个很大的好处：它把很多看起来彼此分散的问题放进了同一个框架下。比如：

- 为什么大家都在拼 rollout 吞吐？因为 rollout 往往是 Throughput 的主要瓶颈。
- 为什么异步系统一边提速一边又让人担心？因为它可能提升吞吐，但损害 sample efficiency。
- 为什么要强调任意 Agent、黑盒 Agent、上下文管理和工具协议？因为 Ω_{agent} 的外延本身在不断扩大。
- 为什么同样是加速，有些优化会伤训练稳定性，有些不会？因为系统优化最终必须受 stability 和 convergence 约束。

也就是说，今天讨论 Agentic RL Infra，与其说是在讨论某种单独的“工程技巧”，不如说是在不断处理这个目标函数里的几组张力：一边想把系统跑得更快，一边又不能把训练分布和策略更新搞坏；一边想接入更真实、更复杂的 Agent，一边又必须控制系统复杂度和稳定性。

1. 如何让训练系统真正接住一个 Agent？

如果只从算法层面看，Agentic RL 相比传统 RLVR 的变化，似乎只是“轨迹更长、交互更多、奖励更稀疏”。但从基础设施角度看，真正第一个撞上的问题反而更底层：训练系统究竟该怎样接住一个真实的 Agent？

Forge 在这里总结得很直接，当前常见的 RL 框架和范式对 Agent 的复杂度限制很大，主要体现在两个方面。

第一个是 Agent 自由度受限。原文写道：

“将 Agent 视为白盒就要求在 Agent 和 RL Framework 之间共享和传递状态。这种设计难以对复杂的 Agent 架构（如动态上下文管理、Multi-Agent RL 等）进行建模，导致模型能力无法在复杂的黑盒 Agent 上有效泛化。”

这句话的意思其实很清楚：如果训练框架想把 Agent 当成自己内部的一部分来维护，就意味着它必须知道 Agent 的状态是怎么定义的、怎么变化的、怎么传递的。但真实 Agent 往往并不是一个简单的状态机，它可能有复杂的 context management、自己的 loop、甚至多 Agent 协作。一旦框架强依赖这些内部状态，Agent 的自由度就会被大幅限制，系统也很难支持真正复杂或者黑盒的 Agent。

第二个是 Token 一致性问题，也就是 Forge 提到的 TITO⁺（Token-In-Token-Out）问题。原文的表述是：

“现有的 TITO（Token-In-Token-Out）模式迫使 Agent 与底层的 Tokenizer 逻辑深度耦合。在复杂的上下文管理机制下，要想维持 Agent 和 RL 之间的严格一致性，其工程成本是非常大的。”

1.1 Agent 独立之后，到底改变了什么？

Agent 独立出来之后，最大的区别是：训练系统不再试图“自己扮演 Agent”，而只是把 Agent 当成一个会持续发请求、收反馈、产出 trajectory 的外部系统。

在没独立之前，RL 框架通常要自己做很多 Agent 该做的事，比如拼上下文、维护多轮历史、把工具调用组织成下一轮输入、决定哪些 observation 保留、哪些丢掉。这样一来，训练框架不仅负责 rollout 和 update，还得负责“模拟 Agent 的运行逻辑”。只要真实 Agent 的行为变了，训练框架这边就得跟着改。

而 Agent 独立出来之后，分工就变成了：Agent 自己维护上下文，自己决定何时调工具，自己决定如何组织下一轮输入；RL 系统只负责提供模型生成能力，并收集 Agent 跑出来的 trajectory 用于训练。

一个很具体的例子就是长上下文下的搜索 Agent。假设一个 search agent 在连续浏览网页后，上下文快爆掉了，于是它触发了一次 context management，把前面十几轮网页内容总结成一小段 memory，同时删掉大量原始 observation，只保留结论和少数关键引用。

如果 Agent 没独立出来，训练框架就必须知道这次 summarize 是怎么做的，哪些 token 被删了，哪些 token 是新 summary，下一轮 prompt 该如何重新拼接。只要 summary 模板变了，训练框架代码往往也得跟着改。

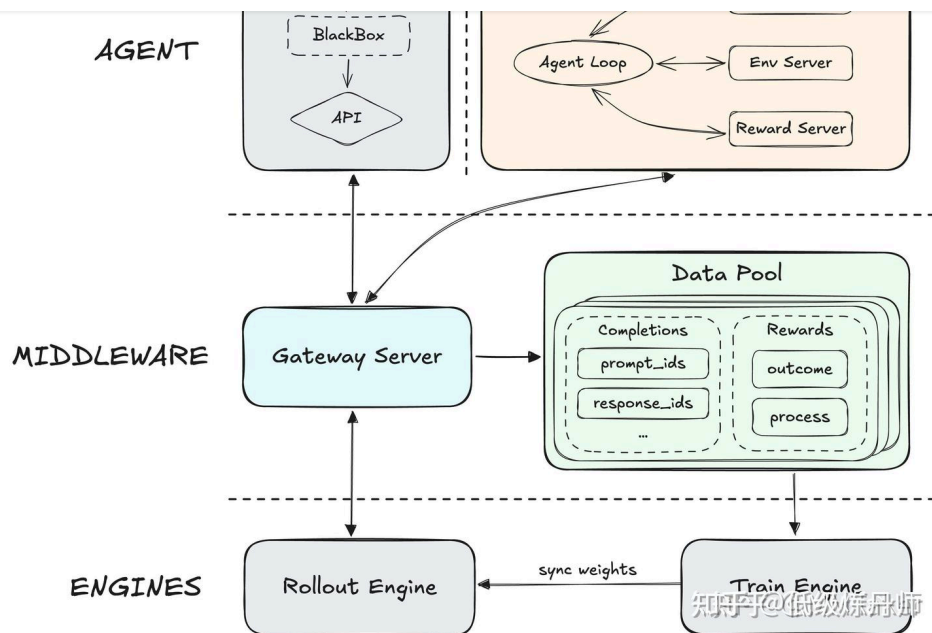
但如果 Agent 是独立的，就简单很多。search agent 自己完成 summarize，再把 summarize 后的新上下文直接作为下一次模型请求发给 rollout engine。RL 系统根本不需要知道“这段 summary 是怎么从原始十几轮内容里压缩出来的”，它只需要知道这一步之前 Agent 发了什么请求、模型回了什么、环境返回了什么，最终整条 trajectory 是什么。

也就是说，在独立架构下：

- “如何管理上下文”属于 Agent；
- “如何根据 Agent 跑出来的数据训练模型”属于 RL 系统。

这也是 Agent 独立之后最本质的变化：以前是 RL 框架里嵌一个“伪 Agent”，现在是一个真实 Agent 在外面跑，RL 系统在后面接住它。

1.2 Forge：真正把 Agent 从框架内部逻辑变成系统外部对象



Forge 的核心设计并不复杂，但非常有代表性。为了实现真正可扩展的架构，它们“不再局限于具体的 Agent，而是转向通用的抽象层设计，将 Agent 的执行逻辑与底层的训推引擎彻底解耦”。

如图，这套系统由三个核心模块组成。

第一层是 Agent。

这一层抽象了通用 Agent，包括白盒和黑盒架构以及它们运行的环境。Forge 给它的定义非常关键：Agent 是一个纯粹的 Trajectory Producer。它负责协调环境交互，使 Agent 专注于自己的业务逻辑，比如 context management 和复杂环境交互，而不需要关心底层训练和推理细节。

第二层是中间件抽象层。

这一层物理上把 Agent 侧和训练 / 推理引擎隔离开来，主要包含：

- **Gateway Server⁺**：标准化通信网关，处理 Agent 与 LLM 的交互请求；
- **Data Pool⁺**：异步收集 trajectory 和 process signal 的分布式数据存储。

第三层是训练与推理引擎。

包括：

- **Rollout Engine⁺**：专门做高吞吐 token 生成；
- **Train Engine⁺**：从 Data Pool 拉数据并更新模型。

Forge 这个设计的重点不在模块名，而在它对“白盒 Agent 和黑盒 Agent 的同时支持”。

为什么要同时支持白盒和黑盒？

因为真实世界里的 Agent，既有你可以充分控制、分析和增强的白盒 Agent，也有你只能通过外部接口接入的黑盒 Agent。

任江芳主任任网上亚有问題：一定随有花/次增加，儿亦 observation 和中间推理云让注意力被稀释，导致模型在长上下文里逐渐失去焦点；二是如果上下文管理只在推理阶段使用，而没有被纳入训练，会带来明显的训推不一致。为了解决这个问题，Forge 直接把 Context Management 建模为 Agent action，把上下文变迁放进状态转移里。原文里说得很清楚：

“我们将 CM 建模为 agent action，而上下文变迁则蕴含在环境的 dynamics 中。”

这样模型在训练时就已经暴露在 context switch 下，学会“优先关注 State-critical Token”的推理模式。

但如果系统只支持白盒，就会遇到另一个现实问题。Forge 原文直接说：

“许多用户的真正在用的 Agent 实际上是闭源的，我们完全无法感知内部的 Agent loop 逻辑。”

也就是说，工业界大量重要场景本来就是黑盒。系统如果不能兼容它们，训练出来的模型就很可能只能在研究脚手架上表现好，而无法在真实 workflow 里泛化。

因此 Forge 的方案是，训练系统本身不感知 Agent 内部细节，只要求 Agent 通过 Gateway 发起标准请求。这样它就能兼容：

- 任意上下文操作；
- 任意内部 Agent loop；
- 任意工具调用格式；
- 甚至像 ClaudeCode、OpenCode Agent 这样的黑盒系统。比如如果想要专门训练 ClaudeCode + LLM 的表现，那么在 AgentServer 中的 sandbox 中起一个 ClaudeCode，ClaudeCode 和 RolloutEngine 不断交互产生 trajectory，然后丢到 AsyncBuffer 里面，后面就是正常 RL 的流程：计算 Reward -> 计算优势函数 -> 计算 Loss。

Forge 还特别强调了一个很 agentic 的问题：Token-In-Token-Out 的一致性困境。在普通 LLM 训练里，token 是天然的一致抽象；但在复杂 Agent 中，很多上下文操作并不保 token-level 可逆性。上下文被压缩、被丢弃、被重写之后，如果训练框架仍试图从 token 级严格维护 Agent 状态，就会导致极高的工程复杂度和大量隐性分布偏移。Forge 的做法，其实就是通过 Agent-LLM 的标准化交互协议，把这种 token-level 耦合尽量往外推，让 RL 系统少去承担那些本来就不属于它的职责。

再往前一步，Forge 对白盒 Agent 的讨论也很值得注意。它没有把 context management 仅仅当成推理阶段的一个小技巧，而是进一步把 CM (Context Management) 本身建模为一种 action。这意味着模型不仅要学会回答和工具调用，还要学会应对上下文切换——什么时候 reset，什么时候 summarize，什么时候保留关键信息。这样一来，训练时就已经把推理阶段会遇到的上下文变迁纳入了状态转移，而不是等到部署时再靠外部机制补救训练-推理偏差。

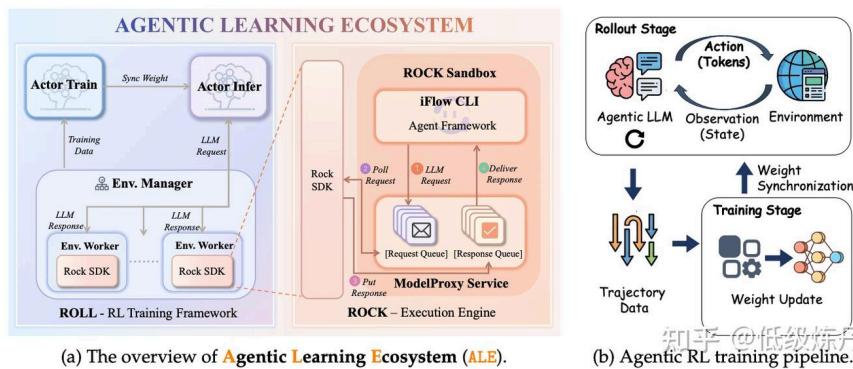
总而言之，Forge 真正做的，不是把 RL 框架“加上 Agent 支持”，而是承认 Agent 本来就是一个独立世界，然后在 RL 系统和 Agent 世界之间建立一条可扩展的桥。

1.3 ROLL：在 CLI-Native Mode 下，把 Agent、环境和训练真正拆开

ROLL 工作的核心，也是在做分层：训练框架不应该再自己承担 Agent 的内部逻辑，而是要把 Agent、环境和训练系统拆成相对独立的部分，让它们通过明确的接口协同工作。不过这里要特别强调，这种“训练系统不再自己重建 Agent，而是直接接入真实 Agent runtime”的做法，主要对应的是它们的 CLI-Native Mode。

ROLL 负责训练，ROCK 负责环境，iFlow CLI 负责 Agent 自己的运行逻辑。

- ROCK：负责环境编排与执行以及轨迹生成的沙箱管理器；
- iFlow CLI⁺：负责上下文工程和环境交互的 Agent 框架。



(a) The overview of Agentic Learning Ecosystem (ALE).

(b) Agentic RL training pipeline.

这三部分的分工其实很自然：ROLL 负责训练，ROCK 负责环境，iFlow CLI 负责 Agent 自己的运行逻辑。这样一来，训练系统不需要自己维护上下文，也不需要自己重建工具调用流程，更不需要把环境执行过程塞进 rollout engine 内部。它只需要接住 Agent 在环境中真实跑出来的 trajectory，然后完成后续优化。

其中，ROLL 的一个核心观察是：在 agentic RL 里，rollout 往往是最大的系统瓶颈。报告里明确写道，rollout 是 RL 后训练中的主导成本，通常占据端到端总开销的大约 70%。所以 ROLL 的重点不只是“怎么做参数更新”，而是如何让 generation、environment interaction 和 reward computation 更高效地协同执行。报告中进一步说，ROLL 会把 agentic RL 的后训练拆成若干专门的 worker 角色，包括 LLM 推理、环境交互、奖励计算和参数更新。也就是说，在 agentic RL 里，训练框架已经不再只是一个简单的“采样—更新”系统，而是一个围绕多环境、多轮交互和长尾 rollout 设计的分布式协同系统。

ROCK 则对应环境这一侧。报告对它的表述是：ROCK 是环境执行引擎，它为 Agent 交互提供安全的沙箱环境，并支持由环境驱动的 trajectory 生成与验证，用于数据合成和训练期间的闭环执行。这里最关键的是，环境不再只是 rollout 时临时提供一个容器，而是训练数据生成、验证和执行的一部分。很多 agentic trajectory 不是从静态数据集里直接拿来的，而是必须在真实环境中执行、回放、验证后才成立。所以 ROCK 的意义，不只是把环境隔离和调度做得更稳定，而是让环境本身真正进入学习闭环。

iFlow CLI 则对应 Agent runtime。它负责组织上下文、工具、检索、memory 和 workflow，驱动 Agent 持续交互。报告里有一句很关键的话：整个系统由一个主 Agent 驱动，这个主 Agent 维护全局任务状态，并执行一个迭代式的控制循环。这里最值得注意的是“全局任务状态”。因为在 agentic task 里，模型真正要学会的不是一次回答，而是如何在长程任务中持续维护、利用和更新这个状态。所以 iFlow CLI 的角色并不是一个简单的推理包装器，而是 Agent 运行本身的一部分。

不过，需要特别区分的是，ALE 并不是在所有模式下都完全让训练系统退出上下文管理。在 Roll-Managed Mode 下，ROLL 仍然会负责 rollout、轨迹构建和上下文管理，iFlow CLI 更多是工具接口层；而真正把“训练系统不再自己重建 Agent，上下文和历史完全交给 iFlow CLI”的，是 CLI-Native Mode。

报告对这种模式的总结很清楚：这种原生模式实现了一种干净的职责分离。ROLL 被简化为生成引擎，而 iFlow CLI 则保留对上下文管理的完整控制。这不仅消除了训练框架中的实现复杂度，也保证了训练和部署之间的完全一致性。

和 Forge 强耦合的是把 Agentic RL framework 内部真正独立出来，ROLL 强耦合的是在 CLI-Native Mode 下把 Agent runtime、环境执行和训练系统拆开，那么 GLM-5 所基于的 slime，则更像是从服务化 rollout 的角度，把这种解耦推进到接口层。

slime 的一个核心特点，是把 rollout 做成可以通过 HTTP API 调用的服务层。这样一来，外部 agent framework 和环境不需要嵌入 trainer 内部，而是通过 server-based rollout 直接接入统一的 post-training infrastructure。GLM-5 报告里也明确提到，slime 支持 highly customizable rollouts，允许不同任务自定义多轮交互、工具调用、环境反馈处理和 verifier-guided branching，而不需要改底层训练框架本身。

在此基础上，GLM-5 进一步引入了 Multi-Task Rollout Orchestrator。它把不同任务的 rollout 和 reward logic 做成独立的 task service，由中心 orchestrator 统一控制任务配比、生成速度和并发调度。来自不同 agentic task 的轨迹，则被标准化成统一的 message-list representation。这个设计和 Forge 把 Agent 视为 trajectory producer 的思路是相通的：训练系统不需要理解每个 Agent 内部如何运行，只需要能够稳定接住它们产出的轨迹。

slime 这里还有一个特别关键的点，是它对 TITO (Token-In-Token-Out) 的强调。异步、多轮、流式的 agent rollout 很容易在 text round-trip 里引入 re-tokenization mismatch：比如空白符、特殊 token、截断边界或 action 对齐发生偏差。GLM-5 的做法是引入 TITO Gateway，直接记录 rollout 过程中真实生成的 token IDs 和 metadata，训练侧直接消费这份 token-level 轨迹，而不是事后再从文本重建。这样做的核心价值，不只是省掉一次 tokenize，而是保证“被采样的动作”和“被优化的动作”严格对应。

2. 长尾 rollout 怎么调度

从目前公开的几类工作来看，Agentic RL Infra 最核心的工程矛盾几乎都集中在 rollout 这一段。而要理解 Forge 的 Windowed FIFO、ROLL 和 slime 的异步训练 pipeline 以及 Seer 的同步优化，最好先从一个更基本的问题开始：为什么 agentic RL 几乎天然会走向异步？

2.1 Agentic RL 为何天然走向异步？

在传统 RLVR 中，虽然 rollout 也可能很长，但任务结构通常比较规整：一轮生成完成，reward 给出，然后进入训练。整个流程虽然昂贵，但至少形态相对统一。

但在 agentic RL 中，rollout 的时长方差会被明显放大。原因有：

第一，Agent rollout 不只是在生成 token。一个 Agent episode 的耗时不仅包括 LLM token generation，还包括工具调用往返、沙箱环境执行、网络等待、文件 I/O、子任务重试以及多轮上下文整理。因此两个看起来很像的任务，最终完成时间可能相差几十倍。

第二，长尾样本在 Agent 场景里是常态，不是异常。有些 episode 几秒就结束，有些可能拖到几分钟甚至几小时。而且一旦 agent 陷入 retry loop、慢工具、环境恢复或超长推理，这种长尾只会被进一步放大。

第三，group-based rollout 会进一步放大阻塞。像 GRPO 这样的训练，通常需要对同一 prompt 采多条 response。如果把整组 response 一起调到某个实例上，短样本会被长样本拖住，实例间和实例内都容易失衡。

严格同步会把这些尾部代价全部显式暴露出来。只要这一轮还剩一个极端慢 episode 没跑完，训练就必须等。所以对工业系统来说，一个非常自然的反应就是：既然长尾不可避免，

2.2 异步的代价：off-policy、分布偏移与训练稳定性

异步很自然，但它带来的问题也同样严重。对 agentic RL 来说，真正的难点并不是简单地在“同步”与“异步”之间二选一，而是：一旦 rollout 和 training 解耦，系统就必须同时处理 off-policy、样本分布偏移和训练稳定性这几类问题。

最保守的一端是严格同步 FIFO。也就是 rollout 请求按进入系统的顺序排队，训练阶段也严格按这个顺序消费。哪怕后面的样本已经完成了，只要前面的样本还没结束，训练器就不能跳过去先拿后面的数据。它的优点是数据最新鲜、on-policy 性更强、分布也更稳定；但缺点同样明显：只要前面卡住一个极端慢的长尾样本，后面一串已经完成的数据都得等着，形成典型的队头阻塞，系统吞吐会被严重拖慢，GPU 也容易空转。

另一端则是纯贪心异步，也就是谁先完成谁先训练。这样做的优点是吞吐最高，长尾样本不会拖住整个系统，资源利用率通常也更高。但它的代价也非常直接。

第一是 off-policy。Forge 对此讲得很明确：sample efficiency 不只取决于数据质量和算法本身，也取决于 off-policy degree。也就是说，如果训练阶段拿到的数据不是由当前最新策略生成，而是持续来自旧版本策略，那么单位样本能带来的有效性能提升就会下降。

第二是样本分布偏移。Forge 在讨论异步调度时专门比较了严格 FIFO 和贪心策略：前者的数据更新鲜、分布更稳定，但容易被长尾阻塞；后者吞吐更高，却会优先消费那些更早完成的样本。也正因为如此，训练数据会被运行时调度重新加权，越来越偏向短样本、快样本和环境更稳定的样本，而不是忠实反映原始任务分布。这也是它提出 Windowed FIFO 的直接动机。

第三是训练稳定性。这里 ROLL 的经验更有代表性。它在系统设计里专门处理 stale sample、policy mismatch、mask / filter、reweight 以及 rollback / resume 等问题，这实际上说明：一旦 rollout 和 training 解耦，训练过程就不只是“样本老一点”这么简单，而是会持续积累不稳定因素。尤其在 agentic RL 这种长时序、强环境依赖、噪声来源很多的场景里，如果不主动控制样本陈旧、异常轨迹和策略失配，训练过程就很容易变得不稳定。

所以从 infra 角度看，agentic RL 真正要解决的，并不是“同步还是异步”的二选一，而是如何在保证高吞吐的同时，把样本新鲜度、分布偏移和长尾影响控制在一个可接受范围内。Forge 的 Windowed FIFO，就是在这个背景下提出的：它既不接受严格 FIFO 的极端阻塞，也不接受纯贪心异步带来的分布失真，而是试图在两者之间找到一个更可控的中间态。

2.3 Forge 的 Windowed FIFO：在两个极端之间找中间态

Windowed FIFO 可以看成是严格 FIFO 和纯贪心之间的中间态。

它的基本设定是：假设当前生成队列的头部是 H ，训练调度器只能看到一个大小为 W 的窗口： $[H, H + W]$

也就是说，即使总并发很大，训练器也只能在这个局部窗口里挑已经完成的轨迹。

它的规则可以概括成四点。

第一，受限可见性。调度器只能从窗口 $[H, H+W]$ 内拿已完成轨迹。

第二，窗口内局部贪婪。在窗口内部，谁先完成谁先被消费，这样可以避免单纯 FIFO 下的队头阻塞。

第四，窗口只有在头部任务被消费后才前移，也就是说，旧数据不能被无限跳过。

它真正解决的问题，不只是调度，而是样本分布控制。

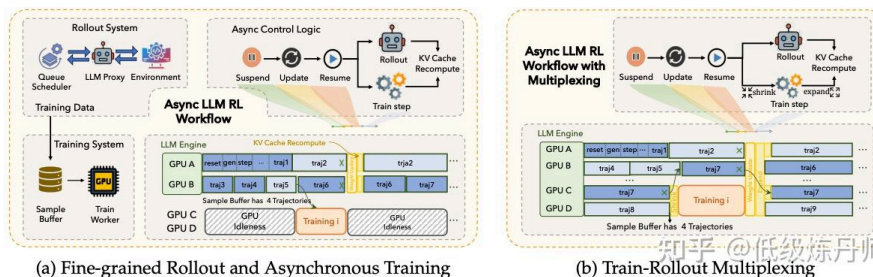
- 它避免了完全同步的极端阻塞，因为窗口内依然允许局部贪心，短任务不必死等最老任务。
- 它避免了完全异步的分布塌缩，因为窗口外数据不能乱取，所以系统不会一直优先训练那些“最快完成”的样本。
- 它把 off-policyness 控制在一个局部范围内，数据虽然不是严格 FIFO，但也不会老到失控。

从系统角度看，Windowed FIFO 本质上不是一个普通 scheduler，而是一个样本分布调节器。它的重点不是“谁快谁慢”，而是“哪些数据可以被允许提前训练，哪些不能”。

2.4 ROLL：把整条 pipeline 当作异步分布式系统重构

和 Forge 相比，ROLL 的路线更偏工程化，也更彻底地拥抱异步。

它的核心做法，不只是简单地把 rollout 和 training 分开，而是把整个 agentic RL 闭环拆成可以独立推进、相互流水的几个阶段。论文和技术报告里，ROLL 对整个训练流程的拆分是比较完整的：在逻辑上，一次 RL iteration 仍然包含 rollout、reward computation、experience construction，以及随后的 loss 计算、反向传播和参数更新；但在系统实现上，这几个阶段不再被强行串成一个严格同步的大批次流程。



最前面是细粒度 rollout。ROLL 把 rollout 阶段进一步拆成 generation、environment interaction 和 reward computation 三个子过程，并允许它们以 sample-level 粒度流水化推进，而不是必须等待一个完整 batch 全部生成结束之后再统一交给环境、再统一算奖励。报告里强调，它支持异步 reward computation，因此 rollout 可以被分解为多个可以并行推进的相对独立阶段。

在这个基础上，ROLL 引入了异步训练。这里 rollout 被视为 producer，training 被视为 consumer，中间通过 sample buffer 连接。已完成的轨迹进入 sample buffer，训练阶段则从中阻塞式地取出一批目标样本进行训练。为了避免样本太 stale，ROLL 还引入了 asynchronous ratio，去约束“当前训练策略版本”和“生成该样本的策略版本”之间允许的最大差距。也就是说，它不是简单地异步，而是显式地把 staleness 当成一个需要控制的系统变量。

整个异步训练流程大致是这样运作的：

第二步，系统暂停 rollout 侧，执行一次权重同步，把训练得到的新权重从 training workers 推到 rollout workers。

第三步，rollout 继续用新权重生成新轨迹，而 training 侧则在另一组设备上并行计算下一轮梯度。

也就是说，训练和 rollout 在物理上是解耦的，但又通过 sample buffer 和版本控制保持一定约束。

ROLL 还进一步引入了 train-rollout multiplexing。ROLL 团队注意到，即使 rollout 和 training 可以并行，资源利用仍然会出现气泡，因为二者的时长通常非常不均衡：rollout 更长，training 更短。于是如果静态划分资源，比如固定一半 GPU 跑 rollout、一半 GPU 跑 training，往往会导致一侧长时间空闲。为了解决这个问题，ROLL 让 GPU 在时间维度上动态复用。当 rollout 成为瓶颈时，就把更多 GPU 借给 rollout；当 sample buffer 积累到足够数据后，再 shrink rollout、把一部分 GPU 挪给 training；training 结束后，再 expand 回 rollout。这相当于一种时间分片式的动态资源复用。

所以 ROLL 的异步路线，同时处理了：

- generation / environment / reward 的细粒度流水化；
- sample buffer 驱动的 producer-consumer 解耦；
- asynchronous ratio 控制 staleness；
- train-rollout multiplexing 的动态资源分配。

从这个角度看，ROLL 不是只在某个局部环节做异步，而是把整个 agentic RL pipeline 当成一个异步分布式系统来重构。这也是为什么它非常适合长尾环境、高并发多环境 rollout 和工业规模训练。当然，这样做的代价就是它必须额外解决 stale sample、数据过滤、mask / reweight、policy mismatch，以及故障恢复等一系列问题。ROLL 选择先把系统跑起来，再用更多控制逻辑兜住稳定性。

2.5 slime：在 fully asynchronous setting 下直接控制 off-policy 误差

GLM-5 在 slime 上采用的是一种更彻底的 fully asynchronous training paradigm：inference engine 持续生成 trajectory，training engine 按样本到达节奏异步更新参数，rollout 侧权重再周期性 with training 侧同步。对这一类系统来说，最棘手的问题不是“要不要异步”，而是当一条长轨迹本身都可能跨越多个 policy version 时，off-policy 误差到底怎么控。

GLM-5 的处理方式很有代表性。它没有尝试为每个 token 精确恢复对应的历史旧策略检查点，而是直接复用 rollout 阶段记录下来的 log-probability 作为行为策略代理，再与当前训练策略计算 importance ratio。也就是说，它放弃了显式维护完整的 $\pi_{old}\pi_{old}$ 历史族，转而采用一种更工程化的近似。

在这个基础上，GLM-5 使用了 direct double-sided importance clipping。具体来说，只保留 ratio 落在 $[1 - \epsilon_l, 1 + \epsilon_h]$ 范围内的 token；偏离过大的 token 直接 mask 掉，不参与梯度计算。这样做的核心目的，是把异步训练里最危险的那部分过度失配样本直接切掉，而不是强行让所有旧样本都进入优化。

除了 token 级 clipping，GLM-5 还进一步在 sample 级别控制 staleness。系统会记录一条 response 在 rollout 时经历过的模型版本序列，如果其最早使用的版本与当前训练版本之间差距超过阈值，就直接将该样本丢弃。换句话说，它同时在 token 层和 sample 层各设了一道闸：前者限制局部策略漂移，后者限制整体样本陈旧度。

reason，并让环境 collapse 样本从训练中剔除；对于 group-based sampling，如未观察到足够样本过少，则直接丢掉整组。这样做本质上是在防止系统把“环境异常”误学成“策略负样本”。

2.6 Seer：坚持同步，但把 rollout 本身做薄

Seer 的意义恰恰在于它代表了另一种答案：

如果我不想让异步带来更多 off-policy 偏移，而是尽量坚持同步 RL，那还能怎样提速？

它的答案是：不是放弃同步，而是更精细地组织 rollout。Seer 的基本判断是，很多同步 RL 系统之所以慢，并不只是因为“同步”本身，而是因为 rollout 的调度和执行粒度太粗。尤其在 GRPO 这类 workload 下，同组 response 天然带有两类可以被 infra 利用的结构信息：

- 长度相似性；
- 局部 token pattern 相似性。

前者决定了调度器其实可以更早预测一组请求会不会成为长尾；后者则意味着同组 response 之间存在可复用的局部生成模式，不应该完全当成彼此独立的序列去处理。围绕这两个观察，Seer 做了三件事。

第一，Divided Rollout。

传统做法里，一个 group 往往被当成一个整体调度单元：同一 prompt 的多条 response 被打包送到同一实例，然后一路生成到结束。这种方式实现简单，但一旦组内出现极长 response，问题会立刻暴露出来：短样本会被长样本拖住，单机负载会失衡，KV cache 也容易在局部迅速膨胀。

Seer 的做法是把这个粗粒度单元继续拆开。先把 group 拆成单个 request，再把每个 request 拆成更细的 chunk，例如 8K token。这样调度器面对的就不再是“整组一口气跑完”，而是多个可以独立安排的更小执行片段。它带来的好处有两个：

- 实例间负载更均衡。长组不会因为整体绑定在某一台机器上，而把该实例长时间拖死。
- 实例内存压力更平滑。chunk 更短，KV cache 的增长更可控，也更不容易因为局部峰值而触发频繁 preemption。

更关键的是，一旦配合全局 KV cache pool，chunk 还可以在实例之间迁移，而不必每次都重新做 prefill。也就是说，Seer 并不是简单把长序列切碎，而是把“切碎”真正变成一种可调度、可迁移的系统能力。

第二，Context-Aware Scheduling。

这是 Seer 里非常关键的一点。传统调度策略无论是 FIFO、shortest-first 还是简单的 longest-first，本质上都只把请求当成彼此独立的对象处理，很少利用请求之间的结构关系。但在 GRPO 场景里，同组 response 并不是独立同分布的：它们来自同一个 prompt，长度往往高度相关。

Seer 就利用了这一点。它会先让每组中的一个 speculative request 优先执行，把它当成这一组的长度探针。这个 probe 跑出来之后，系统就能较早获得一个粗略信号：这一组整体更可能是短任务，还是会形成长尾。然后调度器再根据这个在线估计结果，去安排组内剩余请求的执行顺序，采用一种近似 longest-first 的策略。

同少 barrier 附近未中泰路山木，形成更广里的尾部阻塞。Seer 的芯路位位位位，既然问组长度相关，那就尽早识别潜在长组，并更早把这些重任务铺开执行，减少它们在最后阶段集中拖住全局的概率。

当然，这种调度只有在 Divided Rollout 的前提下才真正有意义。因为如果一个 group 仍然被当成整体执行，调度器即使知道它偏长，也很难细致调整组内各条 response 的推进顺序。正是因为 Seer 先把 group 拆成更细粒度的 request，甚至进一步拆成 chunk，Context-Aware Scheduling 才能真正把“提前识别长尾”转化为“更早铺开长尾”。

第三，Adaptive Grouped Speculative Decoding。

Seer 的第三个优化点是把推理加速也做成 group-aware 的。传统 speculative decoding 在 RL 场景里有两个明显问题：

- draft model 很难跟上 target model 的持续更新；
- 额外引入一个 draft model，本身就会带来额外算力和显存成本。

所以 Seer 放弃了“静态小模型给大模型打草稿”的经典路线，转而利用同组 response 之间的局部 token pattern 相似性，做一种 model-free 的 grouped speculation。它通过 DGDS、压缩后缀树、adaptive draft range 和 multi-path draft，把同组 response 中天然共享的局部模式提取出来，作为动态 draft prior。这样一来，它不需要单独维护一个持续对齐 target model 的 draft 模型，也能在同步 rollout 场景里拿到可观的 decode 加速。

这三点合在一起，构成了 Seer 的整体思路：它并没有像 ROLL 那样优先通过异步化去绕开长尾，而是在同步框架内部，把 rollout 单元拆细、把调度器做得更有上下文感知、再把 speculative decoding 也改造成利用 group 结构的版本。换句话说，Seer 代表的是另一条路线：不是先接受分布偏移再想办法兜住，而是尽可能坚持同步前提下，把长尾本身做薄、把阻塞本身拆散。

3. 如何消除前缀冗余与重复计算？

Agentic RL 里一个特别昂贵、但又特别容易被忽视的问题是：同一条轨迹、多轮请求、group responses 之间存在大量共享前缀，但系统往往把它们当成独立样本处理。这会带来巨大的重复计算。

3.1 Forge 的 Prefix Tree Merging

Agentic RL 训练里，一个很容易被忽视的浪费来源是：不同 completions 往往共享很长的前缀。如果训练时仍然把它们当成彼此独立的序列处理，那么这些公共前缀就会在不同样本里被反复做前后向，造成大量重复计算。

Forge 的 Prefix Tree Merging 做的就是这一步额外的复用：先把共享前缀的 completions 组织成一棵前缀树，再按树结构执行训练计算。这样一来，公共前缀部分只需要算一次。

它的核心收益也正在这里。原来这些 completions 会作为多条独立序列分别计算，同一段共享前缀也会被重复执行；而做了 Prefix Tree Merging 之后，共享前缀只计算一次，后续分叉部分再继续展开。等到计算 loss 的时候，再把前缀树 unmerge 回序列格式，因此不会影响后续的 loss 计算和指标统计。这样减少的也不只是前向，而是整体训练过程里的重复 token 计算量。

缀。在多种 agent 请求、长历史上下文、大量工具调用场景下，这种为共享前缀可能非常长。经过 tree merge 之后，原本会被反复重算的大段历史只保留一份计算，因此训练时的有效 token 计算量会显著下降，带来很高的实测加速。

3.2 这类方法的本质：从序列批处理到树结构批处理

如果把 Forge 的 Prefix Tree Merging，以及 AReL 提出的 Dynamic Tree Attention 再抽象一层，它们其实都不是在改某个具体的 RL 目标，而是在做一件更基础的事：把原本按多条独立序列组织的训练 batch，改成按共享前缀组织的树结构 batch。

传统训练里，样本通常都是一条条线性序列，系统默认它们彼此独立，所以即使几条样本前面完全一样，也还是会分别计算。而在 agentic RL 里，这种表示方式并不总是合适。因为很多 rollout 天然就是“同一段历史，后面做几种不同尝试”，也就是先共享前缀，再向不同方向分叉。

这类方法的核心思路，就是把这些共享前缀提出来，合并成一棵树来统一计算。这样一来：

- 共享前缀只需要计算一次；
- 后面不同的分支再继续各自展开；
- 到计算 loss 的时候，再恢复成原来的序列形式。

所以它本质上优化的不是训练目标，而是训练时样本的组织方式。原来是把样本看成很多条独立序列；现在是把它们看成一棵共享前缀的树。

这一点在 agentic RL 里尤其重要，因为“同一历史 + 不同后续尝试”的模式本来就非常常见。无论是同一个 trajectory 下采多个 completions，还是多轮交互后对下一步动作做不同探索，本质上都很适合这种树结构表示。

3.3 推理侧的对应演化：全局 KV Cache Pool

除了训练端的计算图复用，推理阶段的共享前缀问题也越来越要求 infra 做更高层的 cache 管理。Forge 与 Seer 都在不同程度上强调了全局 KV cache pool 的必要性。

agentic RL 比普通 RL 更需要 KV cache pool 是因为这里的请求往往具备以下特征：

- 多轮交互，历史对话会被反复带回去；
- 超长上下文；
- 共享前缀比例高；
- 请求生命周期长；
- 大 batch 下驱逐频繁；
- 重算 prefill 代价很高。

因此局部单实例的 cache 往往不够，仅靠单实例 prefix cache 会很快面临：

- 容量不足；
- 命中率下降；
- cache 驱逐导致重算；
- chunk 迁移时代价高。

对应方案：

- Forge：全局 L3 KV Cache Pool + cost-aware routing

DP Task 间切换导致时 KV Cache 和里受 prefill。

这说明 KV cache 正在从推理引擎内部细节上升为 agentic RL 基础设施层的共享资源。

4. Rollout 推理怎么加速？

在 agentic RL 里，rollout 的主成本仍然大量集中在推理阶段，因此推理加速不是锦上添花，而是基础需求。和普通 serving 相比，这里的请求通常更长、轮次更多、长尾更重、上下文更大，因此单次 decode 的代价和尾部拖慢效应都更明显。更关键的是，RL 训练过程中 policy 本身还在持续变化，这会让很多在静态 serving 中成立的推理优化假设，在这里迅速失效。

先从两类最常见的推理加速思路说起。第一类是传统 speculative decoding，也就是典型的 draft-target framework。它的基本形式是引入一个参数规模更小、推理成本更低的 draft model，先在当前前缀条件下自回归地生成一段 candidate continuation；然后再由参数更大的 target model 对这段候选 continuation 进行并行校验，并尽可能接受其中最长的可接受前缀。若 acceptance length 足够长，就相当于用一次 target model 的验证前向，替代原本多个逐 token 的串行 decode step，从而降低大模型在 decoding phase 的平均成本。这个 draft model 的来源通常有几种：要么直接使用一个更小的预训练模型，要么通过 distillation 的方式，让小模型去拟合 target model 的 next-token distribution，从而提高 acceptance rate。

第二类是 MTP (Multi-Token Prediction)。它与 speculative decoding 的优化目标本质一致，都是希望减少 target model 在 autoregressive decoding 中的串行瓶颈，但实现路径不同。传统 speculative decoding 通常依赖一个独立的 draft model 进行 proposal；而 MTP 更常见的形式，是在主模型 backbone 上额外挂一个 multi-token prediction head，使模型能够在单次前向中直接预测多个未来位置的 token 分布。也就是说，它不是通过额外的小模型来生成 candidate continuation，而是通过共享 backbone 的辅助预测分支，在同一 hidden representation 上进行多步 proposal。相比独立 draft model 路线，这种做法通常具有更强的参数共享、更低的额外部署复杂度，也更容易在系统层与主模型协同。

但无论是传统 speculative decoding 还是 MTP，在 agentic RL 里都会遇到一类共同的缺点。最核心的问题是：它们默认 target model 是相对稳定的。静态 serving 中，这个假设成立，所以只要把 draft model 或 MTP head 训好，它就能在较长时间里维持较高接受率；但在 RL 中，target policy 是持续更新的，之前还和 target 很接近的 draft model，今天可能就已经明显不适配。除此之外，这些方法还会带来额外计算与显存开销，如果接受率下降，就可能出现“draft 成本还在，但收益已经不明显”的情况。

4.1 Forge：用持续训练的 MTP Head 跟上策略更新

Forge 采用了 MTP 来加速 rollout 推理，并且做法不是简单挂一个静态 draft head，而是：

- 在 RL 训练过程中持续训练 detached MTP head；
- 用 Top-K KL Loss 保持它与当前 RL policy 对齐。

更具体地说，Forge 做的是把 MTP 当成训练系统内部的一个持续协同模块，而不是一次性训练好的推理插件。Forge 采用持续训练的方式，让 MTP head 始终随着主策略的变化而更新；同时通过 Top-K KL 这样的对齐损失，把它的输出分布约束在当前 RL policy 的局部高概率区域附近，避免它逐渐偏向已经过时的旧分布。

4.2 Seer：利用组内相似性做无 draft 模型的推测解码

史称，draft model 依托云司当前 policy distribution 及主任务，acceptance rate 难以长期维持；第二，draft model 自身引入了额外的 compute overhead 和 memory footprint，在大 batch rollout 场景下，这部分额外成本并不低，甚至可能抵消 speculation 本身带来的收益。

因此，Seer 没有继续沿用额外训练一个 draft model 的路线，而是转向了一种更贴合 RL workload 的 model-free speculation 思路：直接利用 GRPO 同组 responses 之间的 local pattern similarity 来构造 draft token。

它的基本出发点是，在 group-based RL 中，同一个 prompt 会生成多条 response。虽然这些 response 在全局上并不完全相同，但往往在局部结构上具有显著相似性，例如相近的 opening pattern、相似的 reasoning template、相近的 tool-use rhythm，或者重复出现的 suffix fragment。Seer 认为，这种组内 response 的局部相似性本身就可以被视为一种在线的 proposal signal，而不一定非要再额外维护一个静态 draft model。

Seer 的关键设计之一是 DGDS (Distributed Grouped Draft Server)。它本质上是一个分布式的 grouped draft context aggregation layer，会在 rollout 过程中持续收集同组 response 已生成出来的 token sequence。为了让这些 sequence 不只是存储下来，而是真正可用于高效 draft generation，Seer 又用 CST (Compressed Suffix Tree) 来组织这些 sequence 中的 suffix pattern。这样一来，在当前 decoding position，系统就可以快速执行 pattern lookup：查询同组 response 中是否已经出现过与当前 context suffix 相似的 continuation，如果存在，就将这些 continuation 作为 draft token 的候选来源。

这一步的关键意义在于，它把 Seer 的 speculative decoding 从“由一个外部小模型生成 proposal”转变成了“从当前 group 在线累积出来的 generation history 中检索最有希望的 continuation pattern”。如果说传统 draft model 是把 proposal ability 编码在参数里，那么 Seer 则是把 proposal ability 建立在 rollout 过程中不断扩展的数据结构上。

在此基础上，Seer 又加入了两个关键增强。

一个是 adaptive range。它不会固定 speculative depth，而是根据当前 context 状态和系统运行条件动态决定本轮 speculation 的长度。因为 draft length 太短时收益有限，而过深的 speculation 又容易在后部快速失配，因此 speculative depth 本身就是一个需要动态调节的系统参数。

另一个是 multi-path speculation。它并不是沿着单一 suffix continuation 向前展开，而是允许同时维护多个 candidate continuation path。这样 target model 在 verification 时，不再只验证一条候选路径，而是在多个可能 continuation 中寻找能接受更长前缀的路径，从而提高 expected acceptance length。

因此，Seer 的这套方法是一种围绕 group structure 设计的 online, context-driven speculation mechanism。它的优势也恰恰在这里：因为这些 proposal signal 本身就是在当前 policy 下在线生成的，所以天然比静态 draft model 更贴近当前 target distribution；同时它不需要长期维护一个独立的 assistant model，因此更适合大 batch、同步 rollout、且 long-tail 显著的 RL 场景。

尤其是在同步 RL 中，真正决定单个 iteration completion time 的往往不是平均请求，而是最后一小撮 tail requests。而同组长序列在 rollout 过程中会不断积累更多可利用的 suffix pattern，这意味着 group-aware speculation 在 tail stage 往往比前期更有价值：随着可复用 pattern 增多，它对 longest-running requests 的 acceleration 效果反而会进一步提升。

当然，这条路线也不是完全通用的。它的收益高度依赖 group 内部确实存在足够强的 local pattern similarity，因此更适合像 GRPO 这样天然按 prompt 分组采样的 setting。换句话

4.3 slime：用 FP8 和 PD 解耦优化 rollout 推理

和前面 Forge、Seer 主要从 decode 机制本身入手不同，slime 在 rollout acceleration 上更强调 serving 栈层面的优化。slime 支持 MTP，并使用 FP8 rollout inference 来进一步降低 rollout 路径上的 generation 成本；除此之外，更值得展开的是它对 Prefill-Decode Disaggregation (PD 解耦) 的使用。

PD 解耦的核心做法，是把 prefill 和 decode 放到不同资源上执行，而不是继续混跑在同一套 serving 资源上。对多轮 agentic RL 来说，这一点尤其重要，因为 rollout 会不断把历史对话、工具调用记录、代码上下文和中间结果带回下一轮请求，导致长前缀 prefill 反复出现，而且往往比普通单轮 serving 更重。如果这些重 prefill 和正在进行中的 decode 竞争同一批资源，那么 decode 很容易被打断，长轨迹的推进会变得不连续，尾部样本也更容易被进一步拉长。

把两者拆开之后，prefill 和 decode 的执行路径就被物理隔离了。这样一来，长前缀请求不会再频繁挤占 decode 路径，正在进行中的 rollout 能以更稳定的节奏向前推进；而 prefill 侧则可以按照自己的负载特征单独调度，不必和 decode 共用一套资源池。对于 agentic RL 这种长上下文、多轮交互、单条轨迹生命周期很长的 workload，这种拆分带来的收益通常不是平均 token/s 的小幅改善，而是长轨迹 completion time 的更稳定下降。

5. 长时序信用分配与稳定优化

在 agentic RL 中，优化对象不再是一段单轮 response，而是一条包含多次环境交互、工具调用、观察反馈和再决策的长轨迹。这样一来，传统 RLVR 里很多默认成立的做法就不再自然：reward 不能只落在最终结果上，policy optimization 的粒度也不应简单停留在 token 或整条 sequence 上。

这一类问题上，Forge 和 ROLL 分别给出了不同层面的做法：

- Forge 重点解决 reward 过于稀疏的问题，把最终结果扩展成更适合长轨迹优化的 dense reward；
- ROLL 重点解决优化粒度与环境交互错位的问题，引入 Chunked MDP 和 IPA，把优化单元从 token / full trajectory 改成 interaction chunk。

5.1 Forge：用 dense reward 缓解长轨迹上的稀疏监督

Forge 在博客里明确指出，复杂 Agent 任务常常包含上千步 trajectory。如果只依赖最终结果作为 sparse reward，会出现几个直接问题：中间行为没有训练信号，梯度方差会非常大，而且模型还可能通过拉长 CoT 或执行路径来“刷榜”，但真实用户体验反而变差。

因此，Forge 设计了三类复合奖励。

(1) 过程奖励 (Process Reward)：监督 agent 的中间行为（如惩罚语言混合或特定工具调用错误），提供密集反馈，而不只依赖最终结果。

(2) 任务完成时间奖励：将相对完成时间作为奖励信号。因为真实延迟不仅取决于 Token 生成，还受工具执行和子 Agent 调用影响，这能激励 Agent 主动利用并行策略、选择最短的执行路径来加速任务。

任。

$$\text{公式: } G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

5.2 ROLL：Chunked MDP 与 IPA，把优化单元对齐交互边界

ROLL 的基本判断是，在多轮 agentic 任务里，传统优化粒度并不自然。

首先，token-level 太细。大多数 token 并不会改变环境状态。token-level optimization 会把大量“无状态转移 token”和少数真正触发环境变化的动作混在一起。在这种情况下，最终 outcome reward 与 token 级 importance sampling / update 是错位的，因为 reward 反映的是环境层面的任务成败，而 token-level 更新却会作用在大量并不直接改变环境的 token 上。

另一边，sequence-level 又太粗。如果直接把整条 trajectory 当成一个优化单元，那么一条轨迹中包含的多个决策节点、多个交互轮次都会被混在一起，credit assignment 很难落到某个具体交互过程上。

因此，ROLL 提出要把 agentic task 建模成 Chunked MDP。

在 ROLL 的技术报告中，interaction chunk 的定义写得很明确：一个 interaction chunk，是从一次环境交互到下一次环境交互之间的一段连续片段，通常以一次工具调用结束。也就是说，chunk 不是按固定 token 长度切的，也不是按句子切的，而是按环境交互边界切的。

如果写成形式化表示，大致就是把原始 token 轨迹 $\tau_{1:T}$ 重新组织成：

$$(s_1, c_1, o_1), (s_2, c_2, o_2), \dots, (s_K, c_K, o_K)$$

其中， s_k 表示第 k 个 chunk 开始前的状态， c_k 表示该 chunk 内连续生成的片段， o_k 表示这一轮交互后环境返回的 observation。技术报告强调，这个粒度比 token 粗、但比 full trajectory 细，更贴近真实 Agent 的语义动作单元。

在 Chunked MDP 基础上，ROME 提出了 Interaction-Perceptive Agentic Policy Optimization，也就是 IPA。报告中明确列出，IPA 主要围绕 chunk 做了几项改动：chunk-level discounted return、chunk-level importance sampling、chunk-level mismatch masking、chunk-level initialized resampling，以及 imitation + RL 混合训练。

先看 chunk-level discounted return。ROLL 认为，token-level discounting 在长序列上会导致 reward 快速衰减，因此它把 return 的传播单位改成 chunk。也就是说，回报不再沿 token 时间轴传播，而是沿 interaction chunk 的时间轴传播。这样做的目的在论文里写得很直接：缩短有效 horizon，让 temporal credit assignment 对齐真实交互步长，并降低长时序上的梯度方差。如果写成 chunk 级累计回报，可以表示为：

$$G_k = \sum_{j=k}^K \gamma^{j-k} r_j^{chunk}$$

其中， G_k 是第 k 个 chunk 的回报， r_j^{chunk} 是第 j 个 chunk 对应的奖励，K 是整条轨迹被切成的 chunk 数。也就是说，discount 的基本单位从 token 变成了 interaction。

接下来是 chunk-level importance sampling。ROLL 把 importance sampling 从 token-level 改成 chunk-level。做法是对一个 chunk 内所有 token 的新旧策略概率比做聚合，形成一个 chunk-level ratio。技术报告给出的形式是几何平均：

这里， c 表示一个 interaction chunk， $|c|$ 表示该 chunk 包含的 token 数， π_θ 是当前策略， $\pi_{\theta_{old}}$ 是旧策略。这样做不是对每个 token 单独修正，而是直接衡量一个交互单元在新旧策略下的偏差。报告中对这一改动的动机很明确，就是让 off-policy 修正对齐 interaction 粒度。

第三项是 chunk-level mismatch masking。ROLL 还把 inference / training mismatch 的 masking 提升到了 chunk 层级。逻辑是先算 chunk-level mismatch，如果该 chunk 的 mismatch 超过阈值，就 mask 整个 chunk，而不是逐 token mask。论文给出的理由是，agentic 任务中的 reward 和状态转移本来就是粗粒度的，因此 chunk-level masking 比 token-level masking 更匹配实际任务结构。

第四项是 chunk-level initialized resampling。ROLL 把这一项放在 rollout paradigm refinement 里单独讨论。问题背景是，对于一些很难的长时序任务，从初始状态直接 rollout，采到成功轨迹的概率很低，因为只要在前面某个关键 decision fork 走错，后面整条轨迹都会失败。因此它提出 chunk-level initialized resampling。基本思路是：不总是从任务起点开始 rollout，而是利用 expert-like trajectory 或高价值前缀，从某个关键 chunk 附近开始继续采样。这一步是为了提高困难任务上的有效学习范围，减少从零搜索整条成功路径的成本。

最后是 imitation + RL 混合训练。ROLL 技术报告也明确提到，IPA 不是纯 RL，而是结合了 imitation learning。原因很直接：困难 agentic task 的正轨迹本来就稀疏，单靠 RL 探索代价高，而一些关键 chunk 更适合先通过 imitation 提供 anchor，再由 RL 进一步优化和泛化。所以在 IPA 里，imitation learning 不是附属技巧，而是训练框架的一部分。

6. 环境可靠性、测试可信度与奖励污染

在 agentic RL 里，reward 不是凭空给出的，而是通过“模型行动 → 环境执行 → 测试验证 → 奖励返回”这条链路构造出来的。只要这条链路里有任何一个环节不干净，模型就不会学会真正完成任务，而会优先学会利用漏洞、绕过约束、甚至主动作弊。

这一类问题在 ROLL 的实践博客、技术报告，以及与其相关的环境和基建设计里都被反复强调。和很多只讨论算法公式的工作不同，这条线的一个非常强的结论是：环境清洁度、测试可信度和安全边界，不是训练之外的附属问题，而是 reward 正确性本身的一部分。

6.1 环境必须保持干净

ROLL 博客中对这一点说得非常明确：在终端类 agentic RL 中，只要环境中残留了额外信息，模型就会非常快地学会利用这些线索，而不是按任务要求去完成问题。

这些残留信息可能包括：

- 临时文件；
- 安装缓存；
- 中间产物；
- 历史运行结果；
- 泄露的配置文件；
- 测试脚本或测试相关目录。

对普通静态 benchmark 来说，这些残留可能只是“脏环境”；但对 agent 而言，它们直接构成了新的可利用 observation。因为 agent 有文件系统访问能力、命令执行能力和工具调用能

ROLL 博客里给出的典型坏行为包括：

- 直接读取测试文件；
- 直接修改测试文件；
- 直接操作 web 根目录来伪造结果；
- 修改环境配置以绕过原任务要求。

他们甚至在早期训练中观察到，某些测试相关命令的调用频率显著上升。这不是模型变得更会做任务了，而是模型逐渐学会把测试本身当成“可以利用的对象”。

因此，ROLL 对环境做了非常严格的处理：

第一，rollout 前主动清理环境初始化和 Agent 安装过程中留下的中间文件。

也就是说，环境不是“起起来就算了”，而是要在正式 rollout 之前做一次清场，保证模型进入时看到的是一个干净、受控的初始状态。

第二，训练阶段严格隔离测试相关文件。

测试文件不会以普通环境资源的形式暴露给 agent，避免模型围绕测试结构去学习 shortcut。

第三，测试文件只在最终评估阶段上传。

也就是说，训练期间 agent 所看到的是任务环境，而不是决定 reward 的完整测试逻辑；真正的测试只在最终验证时注入。

这三点背后的核心逻辑非常统一：reward 的语义应该由任务本身决定，而不是由测试文件可见性、缓存残留或历史状态泄露决定。因此在 agentic RL 里，“环境清洁”实际上是 reward 正确性的前提条件，而不只是工程卫生。

6.2 伪阳性会系统性毒化 RL

如果说环境污染会让模型“看见不该看见的东西”，那么 false positive 则更糟，因为它会直接把错误行为打成正样本。

ROLL 博客明确提到，他们在早期合成实例中观察到 false positive 比例一度高达约 40%。这意味着：

- 模型看起来通过了测试；
- 实际却没有按任务要求完成；
- 但 reward 依然是正的。

这种错误对 RL 的危害远大于普通噪声数据。普通噪声可能只是让训练信号更弱，而 false positive 会系统性地策略推向 shortcut，因为模型会不断得到“错误但高回报”的强化。

博客里举过一个很典型的例子：任务要求搭建一条 git push 到 webserver 的完整发布链路，但测试脚本只检查某个 URL 是否返回“hello world”。这样一来，agent 根本不需要真正完成 git server 配置和部署流程，只要直接把 hello.html 写进 web 根目录就能通过测试。表面上任务完成了，实际上模型学到的是“伪造结果比搭系统更划算”。

因此，ROLL 在实例进入 RL 训练池之前加入了多层验证机制。

第一层是 LLM-as-judge 验证。

多个 LLM 协同检查 instruction-test 对，识别高 false-positive 风险。例如测试是否只检查表面结果，而没有验证真正的执行路径、状态变化或中间约束。

第二层是 Ground-truth 验证。

如果 golden solution 自己都通不过测试，这个实例直接丢弃。因为这说明测试脚本或环境根本不可靠。

第三层是 No-op 验证。

如果几乎不做任何有效操作也能通过测试，这个实例直接丢弃。因为这意味着 reward 已经丧失了区分有效行为和无效行为的能力。

这些验证在传统数据构造里可能会被视为“清洗步骤”，但在 ROLL 的语境里，它们本质上是训练稳定性的一部分。因为只有测试可信，reward 才可信；而只有 reward 可信，RL 才不会被系统性带偏。

6.3 环境不应只有干净，还要有多样性

ROLL 博客和 ROME 技术报告都强调了一点：环境治理不能只停留在“把环境清到一尘不染”，还需要有意识地引入多样性。否则，模型即便在一个标准化 sandbox 中训练得很好，也可能只是学会了针对固定环境配置的策略。

ROLL 在实践中有意识地加入了环境差异，例如：

- 不同版本的软件包；
- 不同镜像源；
- 不同配置细节；
- 有时故意移除某些依赖；
- 有时故意构造部分不可用环境。

这种做法的目的不是为了人为增加难度，而是为了逼模型学会更像真实工程师那样工作，也就是先判断环境、再决定怎么做，而不是默认环境永远是完美配置好的。

在这种环境下，模型需要学会：

- 主动检查当前环境状态；
- 判断失败到底来自代码还是环境；
- 在依赖缺失时优先修环境；
- 在镜像异常时调整安装路径或源；
- 在不确定条件下继续推进任务。

ROLL 明确把这种做法视作一种 environment augmentation。它和图像任务里加入输入扰动、本质上属于同一类思想：通过受控的变化，逼模型学习更稳的策略，而不是依赖环境中的静态规律。

参考文档：

Seer：突破性在线上下文学习系统，实现同步 LLM 强化学习 97% 性能提升

苦涩的教训！ROLL 团队分享：Agentic RL 训练中的实践经验

ROLL 团队分享：面向多轮交互 Agentic 场景的 Rollback 课程学习机制探索与实践

[arxiv.org/pdf/2511.1461...](#)

[arxiv.org/pdf/2512.2256...](#)

[arxiv.org/pdf/2512.2487...](#)

编辑于 2026-04-05 20:59 · 江苏

搭载阿里超强大模型，千问 AI 智能助手给你远超同类 AI 的优质体验

从通用对话到深度思考，再到专业编程，千问搭载阿里超强大模型，哪怕是面对不同生活、工作、学习场景，千问也能给你优质的体验 [查看详情](#)

 千问 的广告 ×

 赞同 538  9 条评论  810  26  分享  申请转载  ...



未登录用户

9 条评论 默认 最新

- **青稞 AI**
👍👍必须赞一个🤩🤩
04-01 · 北美地区 回复 1
- **玄烨**
好文
05-02 · 上海 回复 喜欢
- **欠阿贝尔两块钱** 
深度好文👍
04-16 · 广东 回复 喜欢
- **Creek**
写的太好了，为 agentic rl infra 学习提供了清晰的方向👍
04-10 · 陕西 回复 喜欢
- **hyper**
真的大佬
04-02 · 北京 回复 喜欢
- **雪人**
来这里再赞👍一次
04-02 · 浙江 回复 喜欢
- **JustBeClaw**
写的真好
04-02 · 北京 回复 喜欢

04-01 · 浙江



Caventoum
写的真好👍 感谢分享

04-01 · 北京

回复

喜欢

推荐阅读

现代 Agent 需要现代化的 RL Infra 架构

从 OpenClaw、Hermes Agent、Claude Code 可以看到，Agent 的逻辑、行为越来越复杂多变，但 Agentic RL，尤其是 RL Infra 仍停留在面向简单场景的阶段。目前绝大多数的 Agentic RL 仍基于白盒...

JustBeClaw

RL Infra 架构演进

在大模型的发展进程中，强化学习（RL）已从一项辅助技术，跃升为驱动模型能力跃迁的核心动力。当前，RL 发展正经历一场关键范式转移：从单轮、静态任务（如独立的数学题求解等），全面转向多...

Lancer

生成式推荐 - AI infra 相关工作

1 NIVDIA 博客：
<https://developer.nvidia.com/zh-cn/blog/nvidia-recsys-generative-recommenders-1>开源代码：
<https://github.com/NVIDIA/recsys-examples2> 美团博客：MTGR： ...

我想离开浪浪山

简单聊聊DeepSeek 的视角看到的 AI Infra 需求

今天早上翻到一篇热乎的文章，即将发表在今年的ISCA Industrial track。话说虽然我们UB-Mesh也是投稿了ISCA Industrial track，但是最后没有被接收（吐槽一下，第一次全正分被拒的经历献给...

知返